TREINAMENTO CYPRESS · SERVEREST

DIA 3

Profissionalizando

03

O que vamos fazer hoje

- **3.1** Retomada
- **3.2** Page Objects e arquitetura
- **3.3** Boas práticas e antipadrões
- **3.4** Cucumber
- **3.5** CI/CD: Docker e Jenkins
- **3.6** Relatórios
- **3.7** Banco de dados
- **3.8** Lab final e code review

A pergunta de ontem

Quantos arquivos?

Nos dois primeiros dias:

como faço isso funcionar?

Hoje:

como faço isso durar?

3.2

Page Objects e arquitetura

Passo 1 · Centralizar os seletores

```
// cypress/support/selectors.js
export const login = {
  email: '[data-testid="email"]',
  senha: '[data-testid="senha"]',
  entrar: '[data-testid="entrar"]',
  alerta: '.alert'
}
```

```
import { login } from '../../support/selectors'

cy.get(login.email).type('admin.treinamento@qa.com')
```

Agora é um arquivo

Mas falta metade

Centralizar seletor resolve o "onde estão os elementos".

Falta o **jeito de usar a tela**: a sequência de preencher e-mail, senha e clicar continua repetida em todo lugar.

Um mínimo de classes

```
class LoginPage {
  seletores = { email: '[data-testid="email"]' }   // propriedade

  visitar() {                                      // método
    cy.visit('/login')
    return this
  }
}

export default new LoginPage()
```

`class` o molde · `this` eu mesmo · `new` cria o objeto

Sem herança. Sem `constructor` complexo.

O Page Object

```
class LoginPage {
  seletores = login

  visitar()               { cy.visit('/login'); return this }
  preencherEmail(email)   { cy.get(this.seletores.email).type(email); return this }
  preencherSenha(senha)   { cy.get(this.seletores.senha).type(senha); return this }
  clicarEntrar()          { cy.get(this.seletores.entrar).click(); return this }

  logar(email, senha) {
    this.visitar().preencherEmail(email).preencherSenha(senha).clicarEntrar()
    return this
  }
}
```

`return this`

É o que permite encadear

Antes

```
it('deve exibir erro com senha inválida', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('senhaerrada')
  cy.get('[data-testid="entrar"]').click()
  cy.get('.alert').should('contain', 'Email e/ou senha inválidos')
})
```

Depois

```
it('deve exibir erro com senha inválida', () => {
  LoginPage
    .logar('admin.treinamento@qa.com', 'senhaerrada')
    .validarMensagemErro('Email e/ou senha inválidos')
})
```

A primeira descreve como clicar

A segunda descreve o que está sendo testado

Quando vale

Vale quando:

mais de uma dezena de arquivos · mais de uma pessoa · aplicação que muda

É exagero quando:

cinco testes · você sozinho · tela parada há um ano

Page Object tem custo

Ninguém vai te elogiar por uma arquitetura linda numa suíte de três testes

Comando ou Page Object?

	Para quê	Exemplo
Comando	Ação transversal	`cy.loginViaApi()`
Page Object	Uma tela específica	`LoginPage`

Os dois convivem:

```
beforeEach(() => { cy.loginViaApi() })
it('...', () => { ProdutosPage.cadastrar(produto) })
```

3.3

Boas práticas e antipadrões

O catálogo

#	Antipadrão
1	`cy.wait(3000)`
2	Teste dependente de ordem
3	Seletor por classe CSS ou caminho estrutural
4	Massa fixa em ambiente compartilhado
5	Login pela tela em todo teste

O catálogo

#	Antipadrão
6	Teste sem asserção
7	Um teste gigante com trinta passos
8	Nome vago
9	`if` decidindo o que validar
10	Comentar o teste que falha

Antipadrão 6

Teste sem asserção

Este teste passa

```
it('deve fazer login', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()
})
```

E passa também se o login estiver completamente quebrado

Erro 500? Passa.
Não redirecionou? Passa.

A versão que protege

```
it('deve redirecionar para a home do admin após login válido', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()

  cy.url().should('include', '/admin/home')
  cy.contains('h1', 'Bem Vindo').should('be.visible')
  cy.get('[data-testid="logout"]').should('be.visible')
})
```

Antipadrão 9 · o `if`

```
cy.get('body').then(($body) => {
  if ($body.find('tbody tr').length > 0) {
    cy.get('tbody tr').should('exist')
  } else {
    cy.log('sem produtos')
  }
})
```

Passa com produtos. Passa sem produtos.

Não afirma nada — só descreve o que encontrou.

A versão que afirma

```
cy.intercept('GET', '**/produtos*', {
  fixture: 'produtos-mock.json'
}).as('produtos')

cy.loginViaApi()
cy.visit('/admin/listarprodutos')
cy.wait('@produtos')

cy.get('table.table-striped tbody tr').should('have.length', 2)
```

O teste **cria** a condição que quer validar.

Se você não sabe o que esperar

o problema não é o `if`

é que falta preparar o estado

Testes instáveis · causas

1. Espera por tempo em vez de condição
2. Dependência de estado entre testes
3. Massa compartilhada alterada por terceiros
4. Animação não aguardada
5. Requisição assíncrona não interceptada
6. Elemento que se redesenha entre a busca e a ação

Diagnóstico

```
npx cypress run --spec "caminho/do/teste.cy.js"
```

Rode umas dez vezes.

Falhou 1 em 10? É instável.

Falhou sempre? É bug — e bem mais fácil de resolver.

Conserte em dois dias ou remova da suíte principal

Não há meio termo

Por quê

O motivo não é técnico. É humano.

Teste instável ensina o time a **ignorar o vermelho**.

Quando todo mundo aprende que o vermelho não significa nada, a suíte inteira perde valor — **inclusive os testes bons**.

Um teste instável é pior que teste nenhum

3.4

Cucumber

BDD e Gherkin

BDD é a prática: negócio, desenvolvimento e QA combinando juntos o comportamento esperado, antes do código.

Gherkin é o formato que registra essa combinação, numa linguagem que todo o time lê.

O **Cucumber** faz o Gherkin rodar como teste. Hoje: como ele se encaixa no Cypress.

A feature

```
# language: pt

Funcionalidade: Login na loja

  Cenário: Login com credenciais válidas
    Dado que estou na página de login
    Quando informo o e-mail "admin.treinamento@qa.com" e a senha "treino123"
    E clico no botão entrar
    Então devo visualizar a home do administrador
```

Os steps

```
import { Dado, Quando, E, Entao } from '../../../support/cucumber-pt'
import LoginPage from '../../../pages/LoginPage'

Dado('que estou na página de login', () => {
  LoginPage.visitar()
})

Quando('informo o e-mail {string} e a senha {string}', (email, senha) => {
  LoginPage.preencherEmail(email).preencherSenha(senha)
})
```

A armadilha de versão

```
// Até a v19 — e é o que os tutoriais ensinam
import { Dado, Quando, Entao } from '@badeball/cypress-cucumber-preprocessor'
```

```
(0 , import_cypress_cucumber_preprocessor.Dado)
is not a function
```

Da v20 em diante, só sobraram `Given`, `When`, `Then`.

A solução: poucas linhas

```
// cypress/support/cucumber-pt.js
import { Given, When, Then } from '@badeball/cypress-cucumber-preprocessor'

export const Dado = Given
export const Quando = When
export const Entao = Then
export const E = When
```

O `.feature` continua em português — disso cuida o `# language: pt`.

É o mesmo Page Object de uma hora atrás

O Gherkin descreve o comportamento,
o Page Object sabe usar a tela

Toda biblioteca muda com o tempo

Fixe a versão e leia as notas antes de atualizar

O que faz o BDD render

- Negócio **lê e contribui** com as features
- O cenário nasce **antes** do código, no refinamento
- Linguagem de **negócio**, que qualquer pessoa do time entende
- Features curtas, steps **reaproveitados** e apoiados em Page Objects

O formato não faz o teste sozinho

Asserção, independência e massa própria
valem igual em Gherkin e em JavaScript

O valor está na conversa

O Gherkin registra o que negócio, desenvolvimento e QA combinaram. O Cucumber transforma esse registro numa especificação que **roda**.

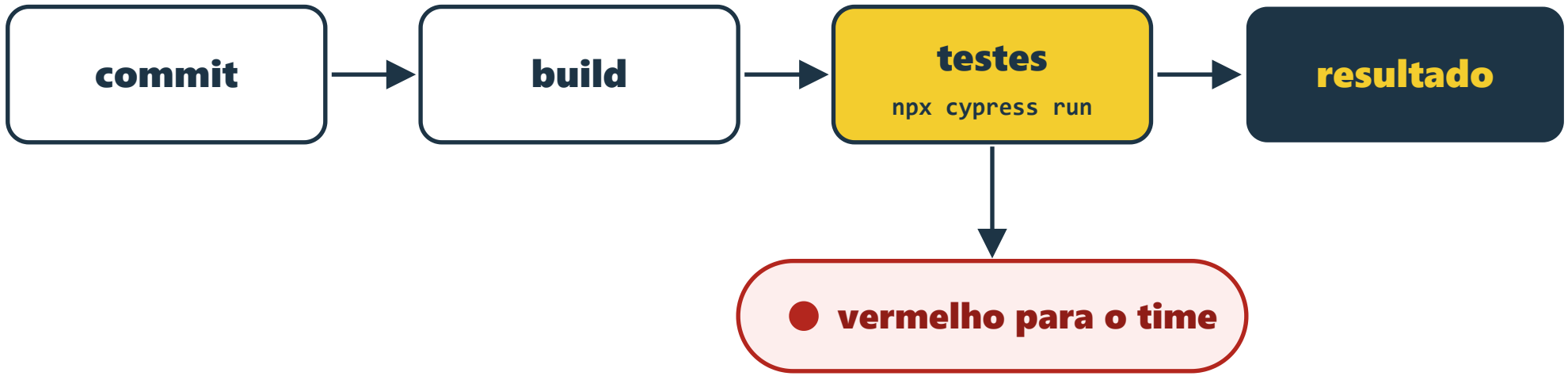
Quando a conversa acontece, o time ganha as duas coisas: acordo sobre o comportamento e um teste que prova esse acordo a cada execução.

3.5

CI/CD

O teste que só roda na sua máquina não protege ninguém

O fluxo



Se quebrar, todo mundo fica sabendo na hora — não duas semanas depois.

O comando que roda no servidor é o mesmo `npx cypress run` que vocês usam desde o Dia 1

Não existe mágica no CI

Fixe a versão

Nunca `latest`

Por quê

Ambiente de CI que muda sozinho gera **falha fantasma**.

O teste quebra e você perde meio dia procurando o que mudou no seu código — quando o que mudou foi a imagem.

E falha fantasma destrói a confiança na suíte mais rápido que qualquer bug.

Aplicação e testes separados

```yaml
services:
  serverest:
    image: paulogoncalvesbh/serverest:3.2.2
    ports: ["3100:3000"]

  cypress:
    build: { context: .., dockerfile: ci/Dockerfile }
    depends_on: [serverest]
    environment:
      - CYPRESS_apiUrl=http://serverest:3000
    command:
      - node scripts/preparar-ambiente.js && npx cypress run --spec "...06-api-usuarios...,...07-api-produtos..."
```

Só os specs de API: a imagem oficial do ServeRest não tem a loja. A do treinamento só roda local porque alguém trocou a URL da API no build dela.

Containerizar depende de a aplicação permitir

Nem toda aplicação permite

Jenkinsfile

```
pipeline {
    agent { docker { image 'cypress/included:15.21.1'; args '--entrypoint=' } }
    environment {
        CYPRESS_senhaAdmin = credentials('serverest-senha-admin')
        CYPRESS_baseUrl = "${env.LOJA_URL ?: 'https://front.serverest.dev'}"
        CYPRESS_apiUrl = "${env.API_URL ?: 'https://serverest.dev'}"
        NO_COLOR = '1'
    }
    stages {
        stage('Instalar dependências') { steps { sh 'npm ci' } }
        stage('Preparar ambiente')     { steps { sh 'node scripts/preparar-ambiente.js' } }
        stage('Executar testes')       { steps { sh 'npx cypress run' } }
    }
}
```

As mesmas etapas, outra máquina

Stage do Jenkins	Etapa que vocês acabaram de ver
`agent { docker ... }`	1 · a imagem `cypress/included`
`Instalar dependências`	1 · o `npm ci`
`Preparar ambiente`	3 · o admin conferido
`Executar testes`	4 e 5 · os `✓` e a tabela
`post { always }`	Guarda vídeo e relatório

O build 1 ficou vermelho

```
  1) Uso 3 — forçar um cenário impossível de reproduzir à mão
       deve expor que a tela não trata o erro 500 do servidor:
     AssertionError: expected '' to include 'undefined'

  ✖  1 of 16 failed (6%)
Finished: FAILURE
```

Na máquina do instrutor, este teste passava sempre. No servidor, **mais lento**, perdeu uma corrida.

O CI achou um teste instável da nossa própria suíte. Corrigido: builds 2 e 3 verdes.

Três detalhes

Detalhe	Por quê
`npm ci`, não `npm install`	Respeita o lock. Ambiente reproduzível
`credentials(...)`	Senha vem do cofre, não do código
`post { always }`	Evidências guardadas **também** quando falha

Falha tem que quebrar o build

CI que não quebra é enfeite

3.6

Relatórios

Relatório é para quem não estava lá

Se precisa de você ao lado
para ser entendido, não está bom

Mochawesome

```
module.exports = defineConfig({
  reporter: 'cypress-mochawesome-reporter',
  reporterOptions: {
    reportDir: 'cypress/reports',
    charts: true,
    embeddedScreenshots: true,
    inlineAssets: true
  }
})
```

```
// cypress/support/e2e.js
import 'cypress-mochawesome-reporter/register'
```

O que mata um relatório

```
× teste 1
× login
× deve funcionar
```

O que quebrou?

```
× deve exibir mensagem de erro ao informar senha inválida
```

Nome de teste não é detalhe de organização

É a interface da sua suíte
com o resto do mundo

3.7

Banco de dados

Para que ir ao banco

1. Conferir o que foi gravado

A tela disse "sucesso". O dado está lá, do jeito certo?

2. Preparar um estado que a tela e a API não deixam criar

Um registro antigo, um cadastro pela metade, um status que só um processo noturno põe

3. Limpar o que o teste criou

E quando não ir

Se a API faz, use a API. Teste que escreve direto no banco fica preso ao formato das tabelas — e ele muda sem avisar o QA.

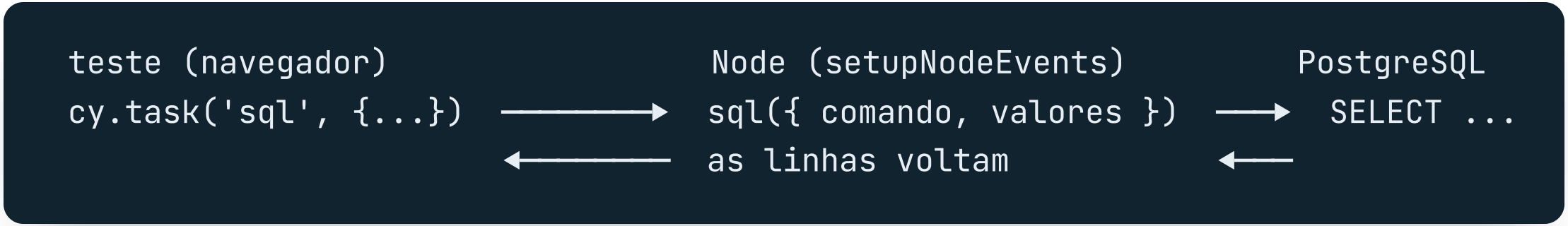
Nunca em produção.

O banco não substitui a tela. Ele complementa.

O navegador não fala com banco

O teste roda dentro do navegador.
Quem abre conexão com banco é o Node.

A ponte: `cy.task`



O teste chama **pelo nome**. A função mora no Node, registrada no `cypress.config.js`.

Sem ela registrada:

```
The task 'sql' was not handled in the setupNodeEvents method.
```

As três peças de uma task

```
// 1. A função, no Node — cypress/tasks/banco.js
async sql({ comando, valores = [] }) { ... return rows }

// 2. O registro — cypress.config.js, dentro do setupNodeEvents
on('task', require('./cypress/tasks/banco'))

// 3. A chamada, no teste — 13-banco-de-dados.cy.js
cy.task('sql', { comando: 'SELECT current_database() AS banco' })
```

O **nome** liga as três: a função `sql` e o `'sql'` do teste.

Faltou uma peça? `The task 'sql' was not handled`.

O contrato da task

Regra	Na prática
Recebe **um** argumento	Vários valores? Um objeto: `{ comando, valores }`
Sempre devolve algo	Sem valor, `return null`. `undefined` dá erro: `returned undefined`
O retorno viaja como JSON	Data chega como texto; função não chega
Erro na task deixa o teste vermelho	Com a mensagem do erro (`ECONNREFUSED ...`)
`console.log` sai no **terminal**	Não no console do navegador
Espera até 60 s	`cy.task('sql', { ... }, { timeout: 120000 })`

Tudo conferido antes da aula.

Para que mais serve uma task

Para **tudo que o navegador não pode fazer**:

- Falar com **banco de dados**
- Ler e gravar **arquivos** no disco
- Usar **bibliotecas do Node** (gerar massa, ler planilha, conferir PDF)
- Preparar ou limpar o ambiente antes do teste
- Escrever no **terminal** do CI, para o log

O teste pede pelo nome; o Node faz; a resposta volta.

Gravar e ler

```
it('deve gravar e ler um cliente', () => {
  const email = `cliente_${Date.now()}@qa.com`
  cy.task('sql', {
    comando: 'INSERT INTO clientes (nome, email) VALUES ($1, $2) RETURNING id',
    valores: ['Cliente do Banco', email]
  }).then(([cliente]) => {
    cy.task('sql', {
      comando: 'SELECT nome, email FROM clientes WHERE id = $1',
      valores: [cliente.id]
    }).its(0).should('deep.equal', { nome: 'Cliente do Banco', email })
  })
})
```

`$1`, nunca `${}`

O SQL e os valores vão **separados**

Colar o valor no texto quebra com `D'Ávila`
— e é a porta do *SQL injection*

O banco da própria loja

```
it('deve gravar no banco o usuário cadastrado pela interface', () => {
  const usuario = { nome: 'Cliente pela Tela', email: `tela_${Date.now()}@qa.com`, password: 'teste123' }

  CadastroUsuarioPage.cadastrar(usuario, { administrador: true })
    .validarMensagem('Cadastro realizado com sucesso')

  // A tela disse "sucesso". O banco diz o que foi gravado de fato.
  cy.task('buscarUsuarioNoBancoDaLoja', usuario.email).should('include', {
    nome: usuario.nome,
    administrador: 'true'
  })
})
```

O ServeRest guarda os dados no **NeDB**: um arquivo, um registro JSON por linha.

As armadilhas do dia a dia

Armadilha	O que fazer
`count(*)` chega `"1"`, `numeric` chega `"10.50"`, data chega em UTC	Compare com texto, ou `count(*)::int` no SQL
O `should` **não repete** o `cy.task`	Espere no Node: `esperarNoBanco`
Consulta lenta: o limite é 60 s	`cy.task('sql', { ... }, { timeout: 120000 })`
`duplicate key value violates unique constraint`	Dado único por execução

Todas conferidas antes da aula.

Dado que chega atrasado

```
// cypress/tasks/banco.js — a espera fica no Node
async esperarNoBanco({ comando, valores = [], segundos = 10 }) {
  const limite = Date.now() + segundos * 1000
  for (;;) {
    const { rows } = await pool.query(comando, valores)
    if (rows.length) return rows
    if (Date.now() > limite) throw new Error(`Nada no banco depois de ${segundos} s: ${comando}`)
    await new Promise((ok) => setTimeout(ok, 500))
  }
}
```

```
cy.task('esperarNoBanco', { comando: 'SELECT nome FROM clientes WHERE email = $1', valores: [email] })
  .its('0.nome').should('eq', 'Cliente do Banco')
```

Para conectar no banco de vocês

- Peça ao DBA: endereço, porta, banco e um **usuário de teste** com permissão só no necessário
- Confira o **acesso de rede**: da máquina e do servidor de CI até o banco
- Senha por **variável de ambiente**: `$env:DB_PASSWORD="..."` antes do `npx cypress open`; no CI, do cofre
- Conexão segura? O `Pool` aceita a opção `ssl`
- **Banco de teste**, nunca produção
- Tarefas com **nome de negócio**: é o Page Object do banco

Muda o banco, muda o driver

O padrão é o mesmo

O driver de cada banco

Banco	Pacote do Node
PostgreSQL	`pg`
SQL Server	`mssql`
MySQL	`mysql2`
Oracle	`oracledb`

A tarefa muda por dentro. O teste continua chamando `cy.task`.

Boas práticas com banco

- Senha por **variável de ambiente**; no CI, do cofre
- Sempre `$1`, nunca `${}` no SQL
- **Banco de teste**, nunca produção
- Uma tarefa com **nome de negócio** (`buscarUsuarioNoBancoDaLoja`) em vez de SQL espalhado pelos testes
- Se a API faz, use a API

3.8

Lab final e code review

Lab final · o enunciado

Parte 1 — Preparação via API

Criar usuário admin · autenticar e obter o token · cadastrar produto

Parte 2 — Interface

Entrar via API · validar na listagem · excluir pela tela

Parte 3 — Data driven

Fixture com três produtos · cadastrar em loop · validar

Parte 4 — Resiliência

Interceptar a listagem · simular erro 500

Requisitos técnicos

- Ao menos **um Page Object**
- Ao menos **dois comandos customizados**
- Ao menos **um alias de requisição**
- **Nenhum** `cy.wait` numérico
- **Nenhum** antipadrão do catálogo
- Relatório Mochawesome gerado

Critério de pronto

`npx cypress run` verde do zero

duas vezes seguidas

Três avisos

Excluir produto não gera mensagem.
A página recarrega e o produto some. Valide a ausência.

Nome de produto é único.
A fixture guarda o modelo; o teste acrescenta sufixo.

A tela não trata o erro 500.
Isso não é defeito do exercício — é o exercício.

A exclusão

```
cy.contains('table.table-striped tbody tr', nomeDoProduto)
  .contains('button', 'Excluir')
  .click()
```

As linhas não têm `data-testid`. A âncora é o nome.

Code review · um arquivo errado de propósito

`cypress/e2e/dia3/codigo-para-revisar.js`

Tem **pelo menos dez** problemas.

Mandem no chat cada um que acharem,
com o item do checklist.

Checklist

- [ ] Os nomes descrevem cenário **e** resultado esperado?
- [ ] Cada teste roda isoladamente?
- [ ] Todo teste tem asserção significativa?
- [ ] Seletores na camada mais alta disponível?
- [ ] Algum `cy.wait` com número?
- [ ] A massa é criada pelo próprio teste?
- [ ] O setup usa o caminho mais rápido?
- [ ] Repetição que deveria ser comando ou Page Object?
- [ ] `if` decidindo o que validar?
- [ ] Segredo **real** no código?

O que tinha no arquivo

No arquivo	Problema	Bloco
`teste 1`, `teste 2`, `teste 3`	Nome vago	3.3 · 3.6
`.form-control` com `.eq(0)`, `.btn-primary`	Seletor por classe e posição	1.7
`cy.wait(3000)`	Espera por tempo	1.5 · 2.2
Login pela tela em cada teste	Setup lento	2.4
O mesmo login copiado duas vezes	Repetição	2.6 · 3.2
`'Mouse Gamer'` fixo	Massa fixa: a 2ª execução falha	2.5

O que tinha no arquivo

No arquivo	Problema	Bloco
Teste 1 cadastra e não confere nada	Sem asserção	3.3
Teste 2 precisa do teste 1	Dependência de ordem	1.6
`if` no teste 2	Nunca falha	3.3
Token de uma pessoa no código	Segredo real, e expira em 10 min	2.5
`DELETE` de id que não existe responde 200	Passa sem excluir nada	2.3
Teste comentado "depois eu vejo"	Dívida silenciosa	3.3

Revisando, você precisa articular por que algo está bom ou ruim

E articular é o que transforma
regra decorada em julgamento próprio

As oito regras

O treinamento em oito frases

1. Teste independente. Sem ordem, sem estado compartilhado.

2. Seletor dedicado a teste. Negocie o `data-testid` com o time.

3. Espere acontecimento, nunca tempo.

4. Massa criada pelo próprio teste.

O treinamento em oito frases

5. Setup pela via mais rápida.

6. Nome de teste que explica o cenário.

7. Não automatize tudo. Automatize o que dói se quebrar.

8. Teste instável se conserta ou se remove. Não há meio termo.

Para onde ir depois

Assunto	Por que vale
Component Testing	Muito mais rápido que E2E
`cypress-axe`	Acessibilidade
Testes visuais	Regressão de layout
Cypress Cloud	Quando a suíte passar de 10 min
TypeScript	Autocompletar nos seus comandos
Real World App	Suíte completa para estudar

Escrevam testes para algo que vocês usam

Uma suíte pequena para um sistema real
ensina mais em duas semanas
que qualquer curso

E quando o teste falhar

Voltem ao método de quatro passos

Era a ferramenta mais útil do primeiro dia.

Continua sendo.

Plano B

Slides de reserva: só se a demonstração ao vivo falhar

Plano B do 3.5 · a execução gravada

Dockerfile

```
FROM cypress/included:15.21.1

WORKDIR /e2e

COPY package*.json ./
RUN npm ci

COPY . .

ENV CYPRESS_baseUrl=https://front.serverest.dev CYPRESS_apiUrl=https://serverest.dev

ENTRYPOINT ["sh", "-c"]
CMD ["node scripts/preparar-ambiente.js && npx cypress run"]
```

Três linhas pedem explicação

`ENV` — o config aponta para a loja local do treinamento, no `localhost`. No container não há loja: no CI, o alvo é o ServeRest público.

`ENTRYPOINT` — a imagem já vem com `ENTRYPOINT ["cypress", "run"]`. Trocamos por um shell para rodar **dois** comandos.

`preparar-ambiente` **antes** — o público é zerado com frequência. O comando de teste continua sendo o `npx cypress run` do Dia 1.

E o `.dockerignore` barra o `node_modules` da sua máquina.

A execução, etapa por etapa

Trechos resumidos da saída real — íntegra em `ci/execucao-registrada.txt`

Um comando

```
docker compose -f ci/docker-compose.yml up --build --abort-on-container-exit
```

Etapa	O que acontece
1	Constrói a imagem dos testes
2	Sobe a API do ServeRest
3	Prepara a massa
4	Roda o `npx cypress run`
5	Devolve o resultado

26 segundos com a imagem já em cache

1 · Constrói a imagem

```
[cypress 1/5] FROM docker.io/cypress/included:15.21.1
[cypress 2/5] WORKDIR /e2e
[cypress 3/5] COPY package*.json ./
[cypress 4/5] RUN npm ci                    CACHED
[cypress 5/5] COPY . .
 cypress  Built
```

Cada linha é **uma linha do Dockerfile**, na ordem.

`CACHED`: nada mudou no `package.json`, então o Docker reaproveita. Na primeira vez, esta etapa leva minutos.

2 · Sobe a API

```
serverest-1  | ServeRest v3.2.2 está em execução
serverest-1  | GET /usuarios 200 21.317 ms - 253
```

Dois containers, **cada linha diz de qual veio**: `serverest-1` ou `cypress-1`.

O `GET /usuarios` é o `healthcheck` do compose perguntando "já está de pé?". Os testes só começam depois do sim.

3 · Prepara a massa

```
cypress-1    | 🔎 Conferindo admin.treinamento@qa.com em http://serverest:3000
cypress-1    | ℹ  Usuário não existe. Criando...
serverest-1  | POST /usuarios 201
cypress-1    | ✅ Pronto. admin.treinamento@qa.com / treino123 — administrador.
```

A API do container **nasce vazia**. Sem o admin, metade dos testes falharia.

`http://serverest:3000` — dentro do compose, o nome do serviço vira endereço.

4 · Os testes batem na API

```
cypress-1    |   API de usuários — escrita
serverest-1  | POST /usuarios 201
cypress-1    |     ✓ deve cadastrar um novo usuário
serverest-1  | POST /usuarios 400
cypress-1    |     ✓ deve impedir cadastro com e-mail duplicado
serverest-1  | POST /usuarios 400
cypress-1    |     ✓ deve recusar cadastro sem os campos obrigatórios
```

Cada `✓` vem **logo depois** da requisição que ele fez. O `400` aqui é sucesso: o teste pediu o erro.

5 · O resultado

```
  Spec                    Tests  Passing  Failing
  ✓  06-api-usuarios.cy.js    6        6        -
  ✓  07-api-produtos.cy.js    5        5        -
  ✓  All specs passed!       11       11        -

cypress-1 exited with code 0
```

A **mesma tabela** do `npx cypress run` do Dia 1.

`code 0` é o que o servidor de CI lê: zero, verde. Qualquer outro número, build vermelho.

O Jenkins, de verdade

Execução real, gravada em `ci/jenkins-execucao-registrada.txt`

Cada linha é um build, cada coluna um stage



O console: os comandos de sempre

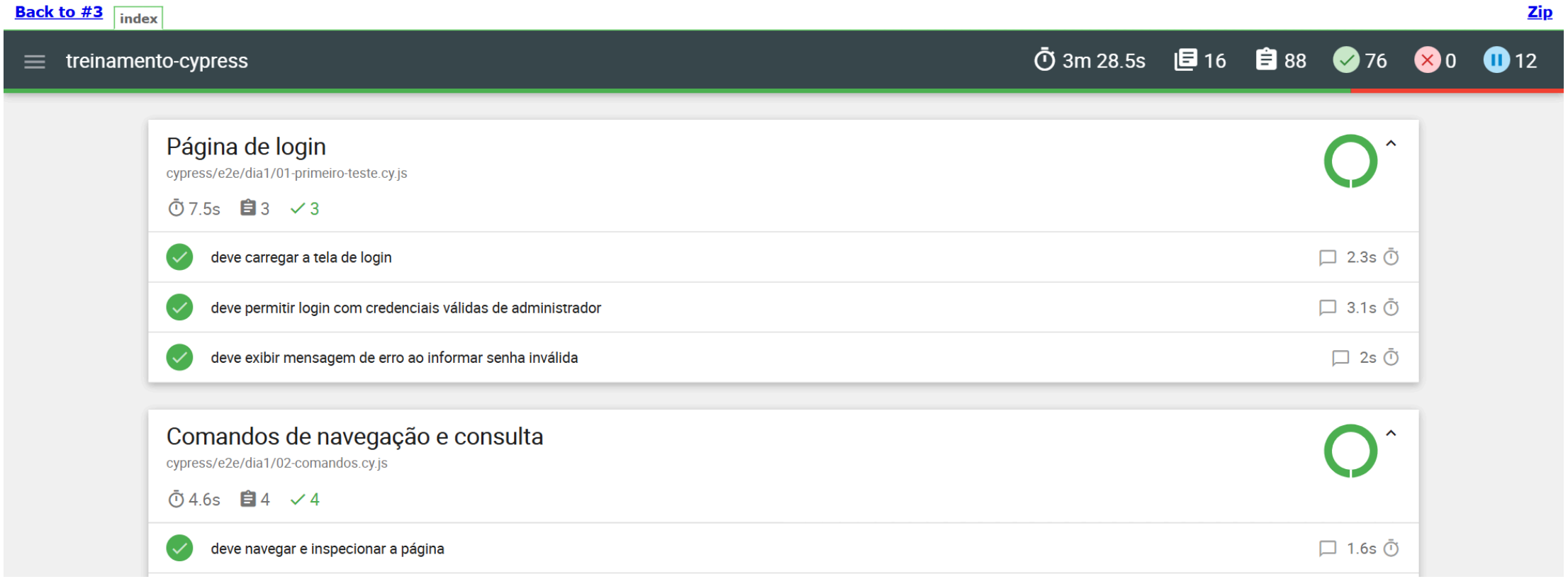
```
[Pipeline] { (Instalar dependências)
+ npm ci

[Pipeline] { (Preparar ambiente)
+ node scripts/preparar-ambiente.js
✅ Usuário já está correto. Nada a fazer.

[Pipeline] { (Executar testes)
+ npx cypress run
  ✓  All specs passed!     02:20     88     76     -     12
```

A linha com `+` é o comando que o Jenkins digitou. **Nenhum é novo.**

O relatório, dentro do Jenkins



O mesmo `cypress/reports/index.html` do Bloco 3.6, com um link em cada build.

Plano B do 3.7 · sem Docker

A tarefa mora no Node · `cypress/tasks/banco.js`

```
const { Pool } = require('pg')

const pool = new Pool({
  host: process.env.DB_HOST || '127.0.0.1',
  port: Number(process.env.DB_PORT || 5433),
  database: process.env.DB_NAME || 'treinamento',
  user: process.env.DB_USER || 'treinamento',
  password: process.env.DB_PASSWORD || 'treino123'
})
```

A senha vem de **variável de ambiente**. O valor depois do `||` é o do banco de estudo; no trabalho, só a variável.

A função que o teste chama

```
module.exports = {
  async sql({ comando, valores = [] }) {
    const { rows } = await pool.query(comando, valores)
    return rows
  }
}
```

E uma linha no `cypress.config.js`, dentro do `setupNodeEvents`:

```
on('task', require('./cypress/tasks/banco'))
```

O primeiro teste

```
it('deve consultar o banco de dados', () => {
  cy.task('sql', { comando: 'SELECT current_database() AS banco' })
    .its('0.banco')
    .should('eq', 'treinamento')
})
```

O resultado é uma **lista de linhas**: `.its('0.banco')` lê a coluna `banco` da primeira.

Quatro verdes

```
PostgreSQL — a conexão pelo cy.task
  ✓ deve consultar o banco de dados
  ✓ deve gravar e ler um cliente
  ✓ deve apagar o cliente criado pelo teste
O banco da própria loja
  ✓ deve gravar no banco o usuário cadastrado pela interface

4 passing (3s)
```

Execução real — íntegra em `banco/execucao-registrada.txt`